



Objective & Lecture Mapping: In Lecture 03, we learned that a mathematically perfect "Table-Driven Agent" is impossible to build due to combinatorial explosion. Instead, we must write Agent Programs. Today, you will implement a **Simple Reflex Agent** using Condition-Action rules, observe its fatal flaws in a partially observable environment, and then upgrade it to a **Model-Based Agent** that maintains an internal memory state.

Part 1: Practical Implementation — Coding the Architectures

Step 1.1: Setting the Trap (Partial Observability)

To see why Simple Reflex agents fail, we must handicap your agent's sensors. We are moving from a Fully Observable world to a Partially Observable one.

1. Open your `visual_grid_game.py` from Lab 01.
2. Locate the `get_percept(self)` method.
3. Modify it so it **no longer returns** the exact global coordinates (`agent_pos`). Instead, it should only return local booleans, e.g., `{'wall_ahead': True/False, 'food_here': True/False}` based on checking the adjacent cell in its current facing direction.

Step 1.2: The Simple Reflex Agent (Implementation & Failure)

1. Create a new class called `SimpleReflexAgent` .
2. Implement a `sense_and_act(self, percept)` method that uses strictly IF-THEN logic (Condition-Action rules). *Do not use an `__init__` method to store history.*
Example Logic: IF `food_here` THEN `suck`; IF `wall_ahead` THEN `turn_left`; ELSE `move_forward`
3. Run the simulation. **Observe:** Watch the agent get trapped in a corner or a U-shaped wall, infinitely repeating a cycle (e.g., turn left, step forward, hit wall, turn left...).

Step 1.3: The Model-Based Agent (Memory & State)

1. Create a new class called `ModelBasedAgent` .
2. Implement an `__init__(self)` method to initialize an internal memory state (e.g., `self.visited_cells = set()` or a relative movement tracker).
3. In `sense_and_act(self, percept)` , first **update the state** (Transition & Sensor Model) by recording the current percept and the last action taken.
4. Update your IF-THEN rules to query the memory.
Example Logic: IF `wall_ahead` AND `left_is_visited` THEN `turn_right`
5. Run the simulation. **Observe:** The agent should now remember where it has been, realize it is in a loop, and choose an alternate path to escape.

Part 2: Theoretical Evaluation & Lecture Mapping

Based on your practical observations above, answer the following questions to map your code directly to the architectural theories covered in Lecture 02.

1. (Remember) According to Lecture 02, why is it impossible to program a mathematically perfect "Table-Driven Agent" for complex environments like Chess? What happens as the agent's lifetime increases?

A Table-Driven Agent is impossible to use for complex environments like Chess because it must store a predefined action for every possible sequence of percepts. As the agent's lifetime increases, the number of possible percept sequences grows exponentially, making the lookup table unimaginably large. Eventually, the table would require impractical amounts of memory and computation, making it impossible to construct or use in real-world problems.

2. (Understand) Look at the code you wrote for your `SimpleReflexAgent`. Identify and explain the specific lines of code that represent the "Condition-Action Rules" discussed in the lecture.

The Condition-Action Rules in the `SimpleReflexAgent` are the `if`, `elif`, and `else` statements inside the `sense_and_act()` method. These rules check the current percept (condition) and immediately choose an action without using any memory.

3. (Analyze) Your `SimpleReflexAgent` likely got stuck in an infinite loop during Step 1.2. Based on the lecture, analyze exactly why this happened. How did the combination of "Partial Observability" and a lack of "Percept History" cause this failure?

The Simple Reflex Agent got stuck in an infinite loop because it makes decisions using only the current percept and has no memory of previous percepts or actions. In a partially observable environment, the agent cannot see the entire grid or know its exact location. It only receives limited information, such as whether there is a wall ahead or food at its current position. When the agent reaches the same type of situation repeatedly (for example, a corner where `wall_ahead = True`), it receives the same percept every time.

4. (Evaluate) In Step 1.3, you added an internal state to your `ModelBasedAgent`. Evaluate how your specific code handles the "Transition Model" (how the world evolves) and the "Sensor Model" (how the agent's actions affect the world).

In my `ModelBasedAgent`, the Transition Model is implemented by updating the agent's internal memory after each action. The agent stores previously observed states in `visited_states` and uses this memory to decide whether to turn left or right when it encounters a wall. This allows the agent to keep track of past experiences instead of reacting only to the current percept. The Sensor Model is implemented through the `sense_and_act(percept)` method. The agent receives percepts such as `wall_ahead` and `food_here` from the environment and uses them to choose an action. For example, if `food_here` is `True`, it performs `Suck`; if `wall_ahead` is `True`, it checks its memory before deciding to `TurnLeft` or `TurnRight`; otherwise, it moves forward. Overall, the Transition Model updates the agent's internal state (memory of visited states), while the Sensor Model interprets the current percepts to determine the most appropriate action. Although the implementation is simple and does not maintain a complete map of the environment, it demonstrates how a Model-Based Agent combines current percepts with stored knowledge to make better decisions than a Simple Reflex Agent.

Finalize and Lock Lab 02 Documentation

